

Relatório - Aula 007

Quiz de abertura: o que cada linha imprime?

Segue o palpite de cada um:

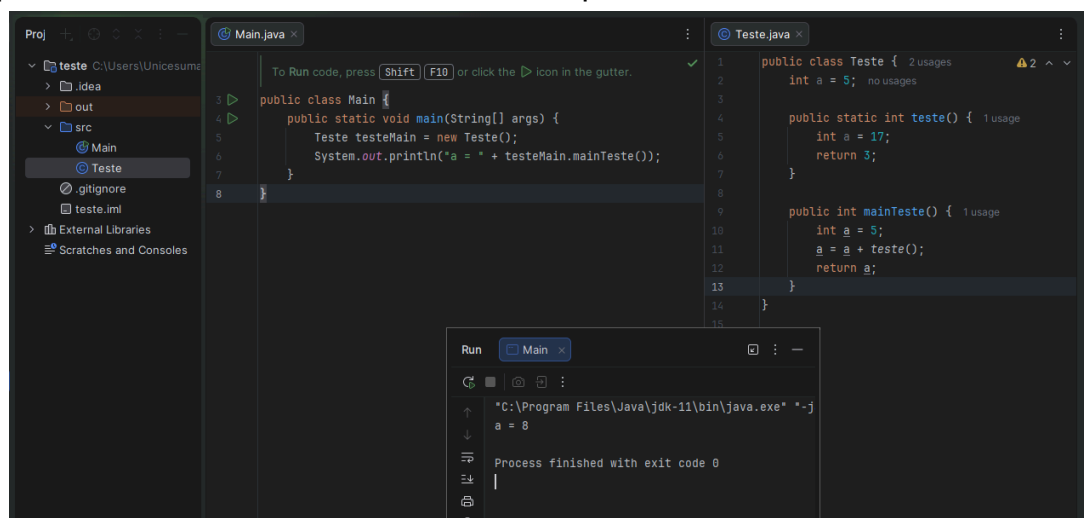
1. 4 64
2. 339
3. Algum erro
4. Algum erro também
5. padrão true
6. false false false

Precedência e associatividade na prática:

- **Precedência:** É quando, sem o uso de parênteses, um operador é analisado antes dos outros independente de posição;
 - Na matemática, a multiplicação tem precedência sobre a soma, por exemplo
- **Associatividade:** É a determinação da ordem em relação aos operadores de mesma precedência, podendo ser da direita para a esquerda ou da esquerda para a direita;

Ordem dos operandos:

- Nem todas as linguagens garantem a ordem dos operandos!
- **Exemplo em Java:** Ele mantém a ordem dos operandos



- Exemplo em C: Ele não mantém a ordem dos operandos

The screenshot shows the OneCompiler C code editor. The code in `Main.c` is as follows:

```

1  #include <stdio.h>
2
3  int a = 5;
4
5  int fun1(void) {
6      a = 17; /* efeito colateral */
7      return 3;
8  }
9
10 int main(void) {
11     a = a + fun1(); /* 8 ou 20? */
12     printf("a = %d\n", a);
13 }
14

```

The console output on the right shows `a = 20`, indicating that the function call `fun1()` modified the global variable `a` before the addition in `main`.

Operadores sobrecarregados:

- Acontece quando um mesmo símbolo tem vários significados.
 - Exemplo: O `+` soma inteiros, reais e ainda concatena, no caso do Java;
 - No Python, é possível ainda atribuir comportamentos para os símbolos como o de adição e multiplicação, podendo modificá-los para realizar essas operações com vetores, por exemplo
- Isso pode levar a comportamentos inesperados, como:

The screenshot shows the OneCompiler Java code editor. The code in `Main.java` is as follows:

```

1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          System.out.println('a' + 1);
6          System.out.println("" + 'a' + 1);
7      }
8  }
9

```

The console output on the right shows `98` and `a1`, demonstrating how the `+` operator is used for both arithmetic and string concatenation in Java.

Conversões de tipos: alargamento, estreitamento e coerção

- **Alargamento:** Quando uma variável tem seu tipo mudado para um de maior capacidade;

- **Estreitamento:** Quando uma variável tem seu tipo mudado para um de menor capacidade, sendo o contrário do alargamento;
 - É uma operação de risco!
- **Coerção:** É uma conversão de tipo automática, presente mais em linguagens fracamente tipadas;
 - Leva a comportamentos inesperados, como esse exemplo em JavaScript:

The screenshot shows the OneCompiler interface with a JavaScript file named 'index.js'. The code in the editor is as follows:

```
1 console.log("5" - 2);
2 console.log("5" + 2);
3 console.log("5" * 2);
4 console.log(true + 1);
5
```

The console output on the right shows the results of these operations:

```
3
52
10
2
```

This demonstrates how JavaScript coerces types: string subtraction results in a number, string addition results in a string, string multiplication results in a number, and boolean addition results in a number.

Expressões relacionais e booleanas:

- Em linguagens como Python, ele aceita, em condicionais como if, vários valores como um booleano;
 - Contam como FALSO em um if:
 - 0 0.0 "" [] {} None
 - Contam como VERDADEIRO:
 - # "0" [0] " "
- Enquanto linguagens como Java, Go, etc; aceitam apenas um booleano;

Avaliação em curto-circuito:

- Curto-circuito é quando uma expressão “composta” é avaliada e, caso o primeiro descumpra a regra, já não se avalia o resto;
- Linguagens baseadas em C fazem isso com && e ||, porém & e | com booleanos não

Sentenças de atribuição:

- Simples:
 - Usa =
- Composta:
 - Tipo o += que é igual à: soma = soma + numero
- Unária:
 - ++
- Expressão:

- Onde a atribuição conta como expressão, devolvendo o valor atribuído, podendo ser usada até no if em algumas linguagens como C e JavaScript;
- Múltipla:
 - Permite atribuição de diferentes variáveis, como em Python:
 - a, b = b, a
- Modo misto:
 - Quando ocorre conflito de tipos;
 - Java, por exemplo, permite o alargamento mas não o estreitamento

The screenshot shows the OneCompiler Java IDE. The language is set to Java. The code in Main.java is as follows:

```

1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          System.out.println("Hello, World!");
6          double d = 3; // ok: alargamento
7          // int i = 3.7; // erro: estreitamento
8          int i = (int) 3.7; // 3 (cast explícito)
9          long l = i; // ok
10         i += 3.7; // COMPILA: cast escondido
11         System.out.println(i);
12     }
13 }
14

```

The console output shows "Hello, World!" followed by the number 6.

- Agora C realiza a coerção:

The screenshot shows the OneCompiler C IDE. The language is set to C. The code in Main.c is as follows:

```

1  #include <stdio.h>
2  int main()
3  {
4      int i = 3.7;
5
6      printf("%d", i);
7  }
8
9

```

The console output shows the number 3.

Estruturas de controle:

Existem várias para facilitar a escrit;

- Sequência:
 - Sentença após a outra;
- Seleção:
 - Escolhe um caminho;

- Aqui pode ocorrer o problema do else pendente, onde problemas de indentação, em linguagens com chaves obrigatórias, podem levar a falsas expectativas de comportamento;
 - E em linguagens sem chaves, com apenas a indentação, pode levar a comportamentos inesperados
- Iteração:
 - Repetição até uma condição;

Seleção múltipla:

- São os casos do switch, por exemplo;
- Em C, se não tiver o break, ele passa em todos os cases!

The screenshot shows the OneCompiler interface with a C file named 'Main.c'. The code is as follows:

```

1  #include <stdio.h>
2  int main()
3  {
4      int index = 1;
5      int odd = 0;
6      int even = 0;
7      switch (index) {
8          case 1:
9              case 3: odd += 1;
10             case 2:
11                 case 4: even += 1;
12                 default: printf("Error\n");
13             }
14
15     printf("%d\n", odd);
16     printf("%d\n", even);
17 }
18
19

```

The console output on the right shows:

```

Error
1
1

```

The AI Agent status bar at the top right indicates a response time of 375 ms.

- em Java, se usar o case como expressão, com o ->, não precisa de break:

Laços controlados por contador:

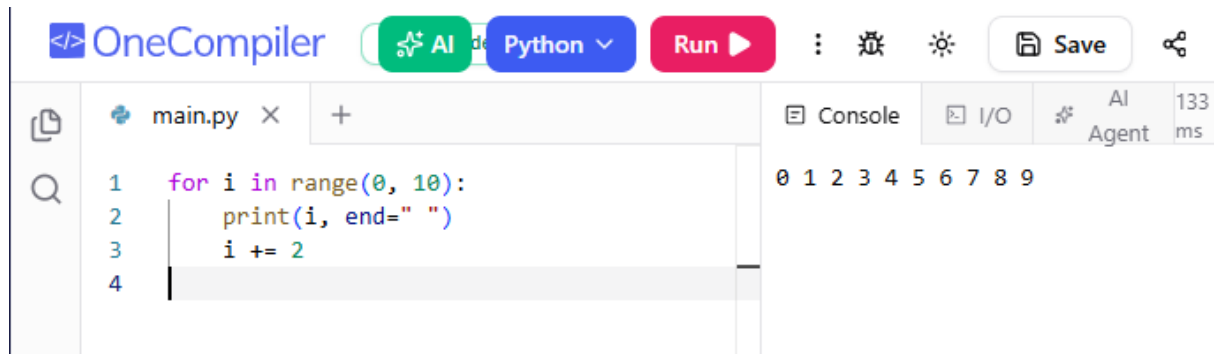
- Tipo o for;
- Em C, é possível alterar o valor do contador dentro do laço



The screenshot shows the OneCompiler interface with the C language selected. The code in `Main.c` is a for loop that iterates from 0 to 9, incrementing by 2, and prints each value. The console output is `0 3 6 9`.

```
1 #include <stdio.h>
2 int main()
3 {
4     for (int i = 0; i < 10; i++) {
5         printf("%d ", i);
6         i += 2;
7     }
8 }
9
```

- Em Python, não dá para mudar



The screenshot shows the OneCompiler interface with the Python language selected. The code in `main.py` is a for loop that iterates from 0 to 9, incrementing by 2, and prints each value. The console output is `0 1 2 3 4 5 6 7 8 9`.

```
1 for i in range(0, 10):
2     print(i, end=" ")
3     i += 2
4
```

Laços lógicos, saídas e iteradores:

- Nos laços lógicos, tem:
 - Pré-teste: como o `while`, que verifica a condição antes de entrar no laço;
 - Pós-teste: Como o `do-while`, que executa no mínimo uma vez;
- Existem saídas posicionadas pelo desenvolvedor, como um `break` rotulado em Java
- Existem iterações baseadas em estruturas de dados, como o `foreach`;
 - Exemplo em JavaScript:



The screenshot shows the OneCompiler interface with the JavaScript language selected. The code in `index.js` is a for loop that iterates over an array of numbers, calculating the total. The console output is `0012` and `60`.

```
1 const notas = [10, 20, 30];
2 let total = 0;
3
4 for (const i in notas)
5     total += i;
6
7 console.log(total);
8 total = 0;
9
10 for (const v of notas)
11     total += v;
12
13 console.log(total);
14
```

Desvio incondicional: o goto

- É um desvio colocado pelo desenvolvedor que pula de um lugar para outro;
- Torna o código difícil de ler;
- Algumas linguagens nem tem, como Java, C, Go;

Comandos protegidos de Dijkstra

- Tipo o if...fi: É como se fosse um teste das guardas do if;